

```
import tkinter as tk # Importa o Tkinter, usado para criar a interface gráfica da aplicação.

from tkinter import PhotoImage # Importa PhotoImage para trabalhar com imagens no Tkinter.

import keyboard # Permite detetar e criar atalhos de teclado globais.

from PIL import Image, ImageTk # Pillow: permite abrir, redimensionar e apresentar imagens.

import pyaudio # Permite gravar e reproduzir áudio através do microfone e das colunas.

import threading # Permite executar tarefas em paralelo sem bloquear a interface gráfica.

from openai import OpenAI # Importa o cliente da API da OpenAI.

import wave # Permite criar e ler ficheiros de áudio no formato WAV.

import io # Permite trabalhar com dados em memória, como se fossem ficheiros.

import json # Permite ler e escrever dados no formato JSON.

import os # Permite trabalhar com ficheiros, pastas e variáveis do sistema operativo.

from dotenv import load_dotenv # Permite carregar variáveis de ambiente a partir de um ficheiro .env.

import wave # Importa novamente o módulo wave; esta segunda importação é redundante.

import pyaudio # Importa novamente o PyAudio; esta segunda importação também é redundante.

import mss # Permite capturar imagens do ecrã/monitores.

import base64 # Permite converter dados binários, como imagens, para Base64.

from io import BytesIO # Permite criar ficheiros virtuais em memória, sem os guardar no disco.

import re # Permite procurar padrões de texto através de expressões regulares.

import matplotlib.pyplot as plt # Permite apresentar imagens, gráficos e pontos numa janela.

import pyautogui # Permite controlar o rato e o teclado através de código.
```

`import replicate` # Permite executar modelos de inteligência artificial através da API da Replicate.

`# Load environment variables` # Comentário que indica que serão carregadas as variáveis de ambiente.

`load_dotenv()` # Lê o ficheiro `.env` e coloca as variáveis disponíveis no ambiente do programa.

`# Audio recording settings` # Define as configurações utilizadas para a gravação de áudio.

`CHUNK = 1024` # Define a quantidade de amostras de áudio processadas de cada vez.

`FORMAT = pyaudio.paInt16` # Define o formato do áudio como inteiros de 16 bits.

`CHANNELS = 1` # Define uma gravação mono, ou seja, apenas um canal de áudio.

`RATE = 16000` # Define uma frequência de amostragem de 16 000 Hz.

`client = OpenAI()` # Cria o cliente da OpenAI, que será utilizado para chamar os serviços da API.

`class ChatbotApp:` # Define a classe principal que contém toda a aplicação.

`def __init__(self, root):` # Método executado automaticamente quando a aplicação é criada.
`self.root = root` # Guarda a janela principal dentro do objeto para poder ser utilizada noutros métodos.

`self.root.title("MYPC")` # Define o título da janela como "MYPC".

`self.root.configure(bg='white')` # Define a cor de fundo da janela como branco.

`# Load saved settings` # Indica que serão carregadas as definições guardadas anteriormente.

`self.settings_file = "settings.json"` # Define o nome do ficheiro onde as configurações serão guardadas.

```
self.settings = self.load_settings() # Carrega as configurações existentes através do método load_settings().
```

```
# Remove overriddenirect to restore taskbar icon # Explica que o overriddenirect não está a ser utilizado.
```

```
# self.root.overriddenirect(True) # Se fosse ativado, removeria a barra de título e alguns elementos normais da janela.
```

```
# Add checkbox for "Always on Top" # Cria a opção para manter a janela sempre à frente das outras.
```

```
self.always_on_top_var = tk.BooleanVar(value=self.settings.get("always_on_top", True))  
# Cria uma variável booleana com o estado guardado ou True por defeito.
```

```
self.always_on_top_checkbox = tk.Checkbutton(root, text="Always on Top",  
variable=self.always_on_top_var, command=self.toggle_always_on_top) # Cria uma caixa de seleção para ativar/desativar a janela sempre no topo.
```

```
self.always_on_top_checkbox.pack(pady=5) # Coloca a caixa de seleção na interface com 5 píxeis de espaço vertical.
```

```
self.root.attributes('-topmost', self.always_on_top_var.get()) # Define se a janela deve permanecer sempre por cima das outras.
```

```
# Load and display the image # Indica que será carregada e apresentada uma imagem.  
script_dir = os.path.dirname(__file__) # Obtém a pasta onde está localizado o ficheiro Python atual.
```

```
image_path = os.path.join(script_dir, "junin.jfif") # Cria o caminho completo para a imagem junin.jfif.
```

```
image = Image.open(image_path) # Abre a imagem utilizando o Pillow.
```

```
image.thumbnail((400, 400), Image.LANCZOS) # Redimensiona a imagem para caber num máximo de 400x400 píxeis.
```

```
self.root.geometry(f'{image.width + 50}x{image.height + 450}+100+100') # Define o tamanho e a posição inicial da janela.
```

```
self.photo = ImageTk.PhotoImage(image) # Converte a imagem do Pillow para um
formato que o Tkinter consegue apresentar.
```

```
self.label = tk.Label(root, image=self.photo, bg='white') # Cria um Label para apresentar
a imagem.
```

```
self.label.pack(side="top", pady=10) # Coloca a imagem no topo da janela com espaço
vertical.
```

```
# Chat display area # Indica a área onde aparecem as mensagens da conversa.
```

```
self.chat_display = tk.Text(root, height=10, width=50, state='disabled', wrap='word',
bg='lightgrey') # Cria uma caixa de texto para apresentar a conversa.
```

```
self.chat_display.pack(pady=10) # Coloca a área da conversa na janela.
```

```
# User input area # Indica a área onde o utilizador escreve mensagens.
```

```
self.user_input = tk.Entry(root, width=50) # Cria uma caixa de texto de uma linha para
introduzir mensagens.
```

```
self.user_input.pack(pady=5) # Coloca a caixa de entrada na interface.
```

```
self.user_input.bind("<Return>", self.send_message) # Faz com que a tecla Enter envie
a mensagem.
```

```
# Voice record button # Indica que será criado o botão para gravação de voz.
```

```
self.record_button = tk.Button(root, text="Click to Record",
command=self.toggle_recording, bg='blue', fg='white') # Cria o botão que inicia ou termina
a gravação.
```

```
self.record_button.pack(pady=5) # Coloca o botão na interface.
```

```
# Register global hotkey for recording # Indica que será criado um atalho de teclado
global.
```

```
keyboard.add_hotkey('ctrl+alt', self.toggle_recording) # Faz com que Ctrl+Alt também
inicie ou pare a gravação.
```

Hear response checkbox # Indica que será criada a opção para ouvir a resposta do chatbot.

```
self.hear_response_var = tk.BooleanVar(value=self.settings.get("hear_response", False))  
# Guarda se a leitura das respostas está ativada.
```

```
self.hear_response_checkbox = tk.Checkbutton(root, text="Hear the response",  
variable=self.hear_response_var) # Cria a caixa de seleção para ouvir as respostas.
```

```
self.hear_response_checkbox.pack(pady=5) # Coloca a caixa de seleção na interface.
```

Voice selection dropdown # Indica que será criada uma lista para selecionar a voz.

```
self.voice_var = tk.StringVar(value=self.settings.get("selected_voice", "alloy")) # Guarda  
a voz selecionada ou utiliza "alloy" por defeito.
```

```
self.voice_label = tk.Label(root, text="Select Voice:") # Cria o texto que identifica a  
seleção da voz.
```

```
self.voice_label.pack(pady=5) # Coloca o texto na interface.
```

```
self.voice_dropdown = tk.OptionMenu(root, self.voice_var, "alloy", "echo", "fable",  
"onyx", "nova", "shimmer") # Cria a lista das vozes disponíveis.
```

```
self.voice_dropdown.pack(pady=5, padx=10) # Coloca a lista na interface.
```

PyAudio setup # Indica que serão preparadas as funcionalidades de áudio.

```
self.p = pyaudio.PyAudio() # Cria o objeto principal do PyAudio.
```

```
self.audio_stream = None # Inicialmente não existe nenhum fluxo de áudio ativo.
```

```
self.is_recording = False # Indica inicialmente que não está a ser feita uma gravação.
```

```
self.chat_history = [] # Cria uma lista vazia para guardar o histórico da conversa.
```

Save settings on exit # Indica que as configurações serão guardadas quando a aplicação fechar.

```
self.root.protocol("WM_DELETE_WINDOW", self.on_closing) # Define o método a  
executar quando o utilizador fecha a janela.
```

```
def load_settings(self): # Define o método responsável por carregar as configurações.
    if os.path.exists(self.settings_file): # Verifica se o ficheiro settings.json existe.
        with open(self.settings_file, 'r') as file: # Abre o ficheiro em modo de leitura.
            return json.load(file) # Lê o JSON e transforma-o num dicionário Python.
    return {} # Se o ficheiro não existir, devolve um dicionário vazio.
```

```
def save_settings(self): # Define o método responsável por guardar as configurações.
    self.settings["selected_voice"] = self.voice_var.get() # Guarda a voz atualmente
    selecionada.
    self.settings["always_on_top"] = self.always_on_top_var.get() # Guarda se a janela deve
    ficar sempre no topo.
    self.settings["hear_response"] = self.hear_response_var.get() # Guarda se as respostas
    devem ser lidas em voz alta.
    with open(self.settings_file, 'w') as file: # Abre ou cria o ficheiro settings.json em modo
    de escrita.
        json.dump(self.settings, file) # Guarda o dicionário como JSON.
```

```
def toggle_always_on_top(self): # Define o método que altera o comportamento "sempre
no topo".
    self.root.attributes('-topmost', self.always_on_top_var.get()) # Atualiza a janela de
    acordo com o estado da caixa de seleção.
```

```
def toggle_recording(self, event=None): # Inicia ou termina uma gravação dependendo
do estado atual.
    if self.is_recording: # Verifica se já está a decorrer uma gravação.
        self.stop_recording() # Para a gravação atual.
```

```
self.record_button.config(text="Click to Record", bg='blue') # Volta a apresentar o botão azul.
```

```
else: # Executa esta parte quando não existe uma gravação ativa.
```

```
self.is_recording = True # Marca a aplicação como estando a gravar.
```

```
self.record_button.config(text="Recording...", bg='red') # Altera o botão para indicar que está a gravar.
```

```
self.start_recording() # Inicia efetivamente a gravação.
```

```
def start_recording(self): # Define o método que prepara a gravação.
```

```
self.frames = [] # Cria uma lista onde serão guardados os pedaços de áudio.
```

```
self.audio_stream = self.p.open(format=FORMAT, # Abre um fluxo de áudio com o formato definido anteriormente.
```

```
channels=CHANNELS, # Define o número de canais de áudio.
```

```
rate=RATE, # Define a frequência de amostragem.
```

```
input=True, # Indica que o áudio vem de um dispositivo de entrada, normalmente o microfone.
```

```
frames_per_buffer=CHUNK) # Define o tamanho de cada bloco de áudio.
```

```
# Start a new thread to handle recording # Indica que a gravação será executada numa thread separada.
```

```
self.recording_thread = threading.Thread(target=self.record) # Cria uma thread que executará o método record().
```

```
self.recording_thread.start() # Inicia a thread de gravação.
```

```
def record(self): # Define o método que recolhe continuamente o áudio do microfone.
```

```
while self.is_recording: # Continua a gravar enquanto is_recording for True.
```

```
data = self.audio_stream.read(CHUNK) # Lê um bloco de áudio do microfone.
```

```
self.frames.append(data) # Adiciona esse bloco à lista de gravação.
```

Stop the audio stream after recording ends # Indica que o fluxo será fechado depois de parar a gravação.

```
self.audio_stream.stop_stream() # Para o fluxo de áudio.
```

```
self.audio_stream.close() # Fecha o fluxo de áudio.
```

Convert recorded audio to bytes # Indica que os pedaços de áudio serão unidos.

```
audio_data = b''.join(self.frames) # Junta todos os blocos de áudio num único conjunto de bytes.
```

```
audio_file = io.BytesIO() # Cria um ficheiro virtual na memória.
```

```
with wave.open(audio_file, 'wb') as wf: # Abre o ficheiro virtual como um ficheiro WAV para escrita.
```

```
    wf.setnchannels(CHANNELS) # Define o número de canais do ficheiro WAV.
```

```
    wf.setsampwidth(self.p.get_sample_size(FORMAT)) # Define o tamanho de cada amostra de áudio.
```

```
    wf.setframerate(RATE) # Define a frequência de amostragem.
```

```
    wf.writeframes(audio_data) # Escreve os dados de áudio no ficheiro WAV.
```

```
audio_file.name = "output.wav" # Dá ao ficheiro virtual um nome para ser aceite pela API.
```

Use OpenAI's speech-to-text # Indica que o áudio será convertido em texto.

```
transcript = self.transcribe_audio(audio_file) # Envia o áudio para transcrição e recebe o texto.
```

```
print("Transcrição", transcript) # Mostra a transcrição na consola.
```

```
if transcript: # Verifica se foi obtido algum texto.
```

```
    self.display_message("You: " + transcript) # Apresenta a transcrição na área de conversa.
```

```
response = self.get_response(transcript) # Envia a transcrição para o chatbot e obtém uma resposta.
```

```
self.display_message("Bot: " + response) # Apresenta a resposta do chatbot.
```

```
if self.hear_response_var.get(): # Verifica se o utilizador ativou a leitura da resposta.
```

```
self.speak_response(response) # Converte a resposta em voz e reproduz o áudio.
```

```
def stop_recording(self): # Define o método utilizado para parar uma gravação.
```

```
self.is_recording = False # Faz com que o ciclo de gravação termine.
```

```
def transcribe_audio(self, audio_file): # Define o método que converte áudio em texto.
```

```
try: # Inicia um bloco para tratar possíveis erros.
```

```
    audio_file.seek(0) # Coloca o cursor do ficheiro no início.
```

```
    response = client.audio.transcriptions.create( # Chama o serviço de transcrição de áudio da OpenAI.
```

```
        model="whisper-1", # Indica o modelo Whisper utilizado para a transcrição.
```

```
        file=audio_file # Envia o ficheiro de áudio para a API.
```

```
) # Termina a chamada à API.
```

```
return response.text # Devolve o texto reconhecido no áudio.
```

```
except Exception as e: # Captura qualquer erro que aconteça durante a transcrição.
```

```
    print("Error during transcription:", e) # Mostra o erro na consola.
```

```
    return "" # Devolve uma string vazia caso a transcrição falhe.
```

```
def get_chat_history(self): # Define o método que obtém o histórico da conversa.
```

```
    return "\n".join(self.chat_history) # Junta todas as mensagens numa única string, separadas por linhas.
```

```
def get_response(self, user_message): # Define o método responsável por obter uma resposta da inteligência artificial.
```

```
    try: # Inicia o tratamento de possíveis erros.
```

```
        response = client.chat.completions.create( # Envia uma mensagem para o modelo da OpenAI.
```

```
            model="gpt-4o-mini", # Define o modelo utilizado para responder.
```

```
            response_format={ "type": "json_object" }, # Obriga a resposta do modelo a ter formato JSON.
```

```
            messages=[ # Cria a lista de mensagens enviada para o modelo.
```

```
                {"role": "system", "content": """"You are a helpful assistant. # Define as instruções principais dadas ao modelo.
```

```
                Your answer must be a JSON. # Obriga o modelo a responder em JSON.
```

```
                If it was a regular question, the type is 'normal'. # Indica que perguntas normais devem ter o tipo normal.
```

```
                If the user ask to click or to point to something, the type is 'click'. # Indica que pedidos para clicar devem ter o tipo click.
```

```
                The click content must be always in english starting with 'point to the...' # Define o formato do texto utilizado para localizar algo no ecrã.
```

```
            Examples of responses: # Indica exemplos de respostas esperadas.
```

```
            {'type': 'normal', 'content': 'The response to the user'} # Exemplo de resposta normal.
```

```
            {'type': 'click', 'content': 'point to the ...'} # Exemplo de resposta que pede uma ação de clique.
```

```
            """"}, # Termina as instruções do sistema.
```

```
            {"role": "assistant", "content": self.get_chat_history()}, # Envia o histórico anterior da conversa.
```

```
            {"role": "user", "content": user_message} # Envia a mensagem atual do utilizador.
```

```
] # Termina a lista de mensagens.
```

```
) # Termina a chamada ao modelo.
```

```
json_response = json.loads(response.choices[0].message.content) # Converte a  
resposta JSON do modelo para um dicionário Python.
```

```
if json_response.get('type') == 'normal': # Verifica se o modelo classificou a resposta  
como normal.
```

```
    return json_response.get('content') # Devolve apenas o conteúdo da resposta.
```

```
else: # Executa esta parte quando o tipo é click.
```

```
    print("Click:", json_response.get('content')) # Mostra na consola o pedido de  
localização.
```

```
    self.click_on(json_response.get('content')) # Envia o pedido para o sistema que  
identifica onde clicar.
```

```
    return json_response.get('content') # Devolve o texto do pedido de clique.
```

```
except Exception as e: # Captura erros ocorridos durante a comunicação com a API.
```

```
    return f"Sorry, I couldn't get a response. Please try again later. Error: {e}" # Devolve  
uma mensagem de erro.
```

```
def speak_response(self, response_text): # Define o método que transforma texto em voz.
```

```
    try: # Inicia o tratamento de possíveis erros.
```

```
        selected_voice = self.voice_var.get() # Obtém a voz selecionada pelo utilizador.
```

```
        response = client.audio.speech.create( # Chama o serviço de conversão de texto para  
voz da OpenAI.
```

```
            model="tts-1", # Define o modelo de text-to-speech.
```

```
            voice=selected_voice, # Define a voz escolhida.
```

```
            input=response_text, # Envia o texto que será convertido em áudio.
```

```
            response_format="wav" # Pede o resultado no formato WAV.
```

```
) # Termina a chamada à API.

audio_data = response.content # Obtém os bytes do áudio devolvido pela API.
audio_file = io.BytesIO(audio_data) # Cria um ficheiro virtual com o áudio.
audio_file.seek(0) # Coloca o cursor no início do áudio.

# Play the WAV audio using PyAudio # Indica que o áudio será reproduzido pelo
PyAudio.

wf = wave.open(audio_file, 'rb') # Abre o áudio WAV para leitura.
p = pyaudio.PyAudio() # Cria um novo objeto PyAudio.

stream = p.open(format=p.get_format_from_width(wf.getsampwidth()), # Define o
formato utilizado para reproduzir o áudio.

                channels=wf.getnchannels(), # Obtém o número de canais do áudio.
                rate=wf.getframerate(), # Obtém a frequência de amostragem do áudio.
                output=True) # Indica que este fluxo será utilizado para saída de áudio.

data = wf.readframes(CHUNK) # Lê o primeiro bloco do áudio.
while data: # Continua enquanto existirem dados de áudio.
    stream.write(data) # Envia o bloco de áudio para as colunas.
    data = wf.readframes(CHUNK) # Lê o bloco seguinte.

stream.stop_stream() # Para a reprodução do áudio.
stream.close() # Fecha o fluxo de áudio.
p.terminate() # Liberta os recursos utilizados pelo PyAudio.
except Exception as e: # Captura qualquer erro durante a reprodução.
    print("Error during text-to-speech:", e) # Mostra o erro na consola.

def display_message(self, message): # Define o método responsável por apresentar
mensagens no chat.
```

`self.chat_display.config(state='normal')` # Permite temporariamente escrever na caixa de texto.

`self.chat_display.insert(tk.END, message + "\n")` # Adiciona a mensagem no final da conversa.

`self.chat_history.append(message)` # Guarda a mensagem no histórico.

`self.chat_display.config(state='disabled')` # Volta a bloquear a edição manual da caixa de conversa.

`self.chat_display.see(tk.END)` # Faz scroll automático para mostrar a mensagem mais recente.

`def send_message(self, event=None):` # Define o método que envia uma mensagem escrita pelo utilizador.

`user_message = self.user_input.get().strip()` # Obtém o texto escrito e remove espaços no início e no fim.

`if user_message:` # Verifica se o utilizador escreveu alguma coisa.

`self.display_message("You: " + user_message)` # Apresenta a mensagem do utilizador.

`self.user_input.delete(0, tk.END)` # Limpa a caixa de texto depois de enviar.

`response = self.get_response(user_message)` # Obtém a resposta da inteligência artificial.

`self.display_message("Bot: " + response)` # Apresenta a resposta na interface.

`if self.hear_response_var.get():` # Verifica se a leitura em voz alta está ativada.

`self.speak_response(response)` # Reproduz a resposta em áudio.

`def on_closing(self):` # Define o que acontece quando a aplicação vai ser fechada.

`keyboard.unhook_all_hotkeys()` # Remove todos os atalhos de teclado globais registados.

`self.save_settings()` # Guarda as configurações atuais.

`self.root.destroy()` # Fecha a janela principal da aplicação.

def click_on(self, click_this): # Define o método que identifica algo no segundo monitor e clica nesse local.

segundo_monitor_pixels = 1920 # Define que o segundo monitor começa 1920 píxeis depois do primeiro no eixo X.

image = self.capture_and_show_image_from_second_monitor(1920, 1080) # Captura uma imagem de 1920x1080 do segundo monitor.

buffered = BytesIO() # Cria um ficheiro virtual em memória para guardar a imagem.

image.save(buffered, format="PNG") # Guarda a imagem capturada no buffer como PNG.

base64_image = base64.b64encode(buffered.getvalue()).decode('utf-8') # Converte a imagem para Base64 para poder ser enviada para a API.

print("Chamando replicate") # Informa na consola que a API Replicate será chamada.

Run the model and get the output # Indica que será executado um modelo de inteligência artificial.

output = replicate.run(# Executa o modelo através da Replicate.

"zsxkib/molmo-7b:76ebd700864218a4ca97ac1ccff068be7222272859f9ea2ae1dd4ac073fa8de8", #
Identifica especificamente o modelo e a versão utilizados.

input={ # Cria os parâmetros enviados para o modelo.

"text": click_this, # Envia a instrução sobre aquilo que deve ser localizado.

"image": "data:image/png;base64," + base64_image, # Envia a captura de ecrã como uma imagem Base64.

"top_k": 100, # Define o número de candidatos considerados pelo modelo.

"top_p": 1, # Define o limite de probabilidade utilizado na geração.

"temperature": 1, # Define o nível de aleatoriedade da resposta do modelo.

"length_penalty": 1, # Define a penalização relacionada com o comprimento da resposta.

"max_new_tokens": 1000 # Define o número máximo de tokens que o modelo pode gerar.

} # Termina os parâmetros de entrada do modelo.

) # Termina a execução do modelo.

Print the output to check its structure # Indica que o resultado será mostrado para análise.

print("Output:", output) # Mostra na consola a resposta devolvida pelo modelo.

Pattern to extract x and y coordinates from the output, whether in <point> or <points> format # Define uma expressão para encontrar coordenadas na resposta.

pattern = r'x\d*"([\d.]*)" y\d*"([\d.]*)" # Procura valores numéricos associados aos atributos x e y.

matches = re.findall(pattern, output) # Procura todas as coordenadas existentes no resultado.

Convert the extracted strings into float values and store as a list of tuples # Converte as coordenadas encontradas.

coordinates = [(float(x), float(y)) for x, y in matches] # Converte cada coordenada de texto para números decimais.

Print the coordinates # Indica que as coordenadas serão apresentadas na consola.

print("Coordinates of points:") # Mostra um título para a lista de coordenadas.

for i, (x, y) in enumerate(coordinates): # Percorre todas as coordenadas encontradas.

print(f"Point {i + 1}: ({x + segundo_monitor_pixels}, {y})") # Apresenta as coordenadas ajustadas à posição do segundo monitor.

Display the image and the points # Prepara uma visualização da imagem e dos pontos encontrados.

```
fig, ax = plt.subplots() # Cria uma figura e um eixo para apresentar a imagem.
```

```
ax.imshow(image) # Apresenta a captura de ecrã dentro do gráfico.
```

```
# Convert coordinates from relative to pixel positions # Converte percentagens em posições reais em píxeis.
```

```
width, height = image.size # Obtém a largura e altura da imagem capturada.
```

```
x_coords = [x / 100 * width for x, y in coordinates] # Converte cada coordenada X percentual para píxeis.
```

```
y_coords = [y / 100 * height for x, y in coordinates] # Converte cada coordenada Y percentual para píxeis.
```

```
# After extracting the coordinates # Indica que os pontos serão utilizados para controlar o rato.
```

```
for x, y in zip(x_coords, y_coords): # Percorre simultaneamente as coordenadas X e Y.
```

```
    pyautogui.moveTo(x + segundo_monitor_pixels, y, duration=0.5) # Move o rato para a posição encontrada durante meio segundo.
```

```
    pyautogui.click() # Clica na posição encontrada.
```

```
# Plot points on the image # Define se os pontos serão apresentados visualmente na imagem.
```

```
plotar = False # Desativa a apresentação gráfica dos pontos.
```

```
if plotar: # Só executa o código seguinte se plotar for True.
```

```
    ax.scatter(x_coords, y_coords, color='black', s=400, marker='o') # Desenha círculos pretos nos locais encontrados.
```

```
print("coords", x_coords, y_coords, image.size) # Mostra as coordenadas e o tamanho da imagem na consola.
```

with mss.mss() as sct: # Cria um objeto MSS para obter informações sobre os monitores.

Get the second monitor's position # Indica que será obtida a posição do segundo monitor.

second_monitor = sct.monitors[2] # Obtém os dados do segundo monitor através do índice 2.

x_pos, y_pos = second_monitor["left"], second_monitor["top"] # Obtém as coordenadas X e Y do segundo monitor.

Set the figure position on the second monitor # Define onde a janela do gráfico será apresentada.

fig.canvas.manager.window.wm_geometry(f"+{x_pos}+{y_pos + 600}") # Posiciona a janela do gráfico no segundo monitor.

Show plot with points # Indica que o gráfico será apresentado.

plt.title("Coordinates on Image") # Define o título da janela/gráfico.

plt.show() # Mostra a imagem com os pontos encontrados.

return output # Devolve o resultado original produzido pelo modelo.

def capture_and_show_image_from_second_monitor(self, width=400, height=400): # Define o método que captura uma região do segundo monitor.

Initialize mss for screen capturing # Indica que será inicializada a captura do ecrã.

with mss.mss() as sct: # Cria um objeto MSS para capturar o ecrã.

Get the monitor information # Obtém informação sobre os monitores ligados.

monitors = sct.monitors # Guarda a lista de monitores detetados pelo MSS.

Check if there's a second monitor available # Verifica se existe um segundo monitor.

if len(monitors) < 2: # Verifica se existem pelo menos dois elementos na lista de monitores.

raise ValueError("No second monitor detected.") # Interrompe o programa com um erro caso não exista um segundo monitor.

Monitor 2 is at index 2 (index 1 is the primary monitor) # Explica a estrutura da lista de monitores utilizada pelo MSS.

second_monitor = monitors[2] # Obtém o segundo monitor através do índice 2.

Define the region to capture: 400x400 starting from the top-left corner of the second monitor # Define a zona que será capturada.

capture_region = { # Cria um dicionário com a posição e o tamanho da área a capturar.

"top": second_monitor["top"], # Define a coordenada vertical inicial da captura.

"left": second_monitor["left"], # Define a coordenada horizontal inicial da captura.

"width": width, # Define a largura da área capturada.

"height": height # Define a altura da área capturada.

} # Termina a definição da região de captura.

Capture the screen in the defined region # Indica que a captura será efetuada.

screenshot = sct.grab(capture_region) # Captura a região definida do segundo monitor.

Convert it to a PIL Image # Converte a captura para uma imagem utilizável pelo Pillow.

return Image.frombytes("RGB", screenshot.size, screenshot.rgb) # Cria e devolve uma imagem RGB a partir dos dados capturados.

```
if __name__ == "__main__": # Verifica se este ficheiro está a ser executado diretamente e não importado por outro programa.
```

```
    root = tk.Tk() # Cria a janela principal do Tkinter.
```

```
    app = ChatbotApp(root) # Cria uma instância da aplicação e inicializa toda a interface.
```

```
    root.mainloop() # Inicia o ciclo principal do Tkinter, mantendo a aplicação aberta e a responder aos eventos.
```